

Lab 6: Drone UART Telemetry & Ground Station

ECE 3301 – Microcontrollers | Fall 2026

Instructor: Sasoun Torousian

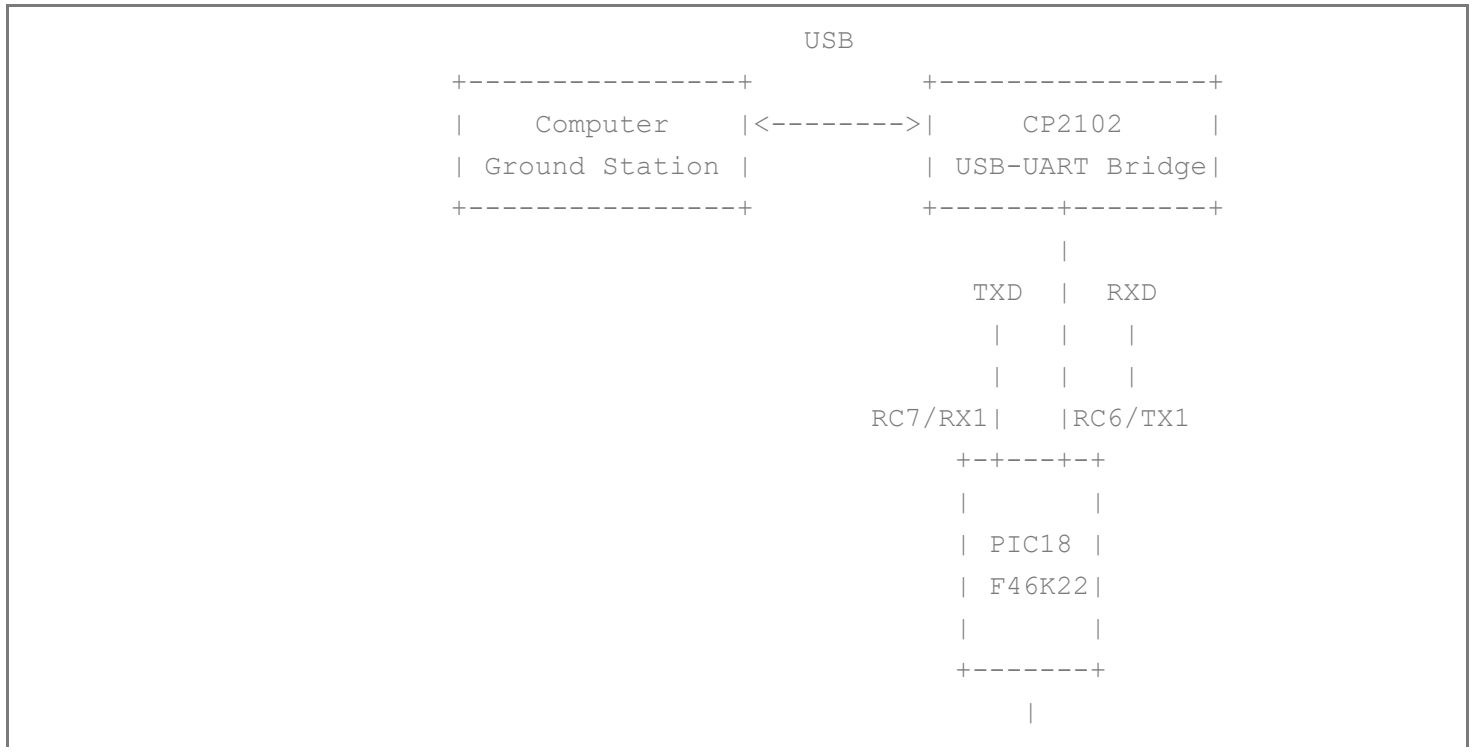
Lab 6:Drone UART Telemetry & Ground Station

1. Introduction

In the previous labs, you built the beginning of a drone safety supervisor using digital I/O, timers, state machines, and interrupts. Your controller could monitor inputs, maintain operating states, and perform several activities without stopping the processor with long delays. Until now, however, the drone controller has operated mostly by itself. A real flight controller must communicate with other systems. It may send battery information, altitude, flight state, sensor measurements, warnings, and diagnostic information to another processor or to a ground-control computer. It must also be able to receive commands. In this lab, you will add your first real communication interface to the drone controller. The PIC18F46K22 will communicate with a computer using its EUSART1 hardware and a CP2102 USB-to-TTL serial adapter. The PIC will act as a simplified drone flight controller, while the computer serial terminal will act as the ground station. The UART lecture introduced the PIC18F46K22 EUSART, including TXSTA1, RCSTA1, BAUDCON1, SPBRG1, TXREG1, RCREG1, TX1IF, and RC1IF. It also showed that UART hardware performs the actual bit-level transmission after the CPU places a byte into TXREG1, while received data is placed into RCREG1. Your job in this lab is to turn those individual UART operations into a small communication system. By the end of the lab, your computer should behave like a simple drone ground station. It will continuously receive telemetry from the PIC and will also be able to send commands back to the PIC.

2. What You Are Building

You will build the following communication system:



```
                                                    Drone Controller
```

The CP2102 converts between the computer's USB interface and the UART interface used by the PIC. TX and RX must be crossed: PIC RC6/TX1 connects to CP2102 RXD, and PIC RC7/RX1 connects to CP2102 TXD. Both devices must also share GND. This is the same communication path introduced in the UART lecture. Your system will perform two operations simultaneously. First, the PIC will periodically transmit drone telemetry to the computer. Second, the PIC will monitor incoming UART commands and respond to them without stopping the rest of the firmware. The finished system should behave conceptually like this:

```
PIC18F46K22                          Ground Station

$STATE,SAFE ------------------------>
$TIME,1 ---------------------------->
$STATE,SAFE ------------------------>
$TIME,2 ---------------------------->

                        <----------- S

$STATE,SAFE ------------------------>
$TIME,2 ---------------------------->
$ACK,STATUS ------------------------>

                        <----------- A

$ACK,ARM ---------------------------->
$STATE,ARMED ------------------------>

                        <----------- D

$ACK,DISARM ------------------------>
$STATE,SAFE ------------------------>
```

This is no longer an UART echo demonstration. You are creating a very small communication protocol between a flight controller and its ground station.

3. Learning Objectives

After completing this lab, you should be able to configure EUSART1 on the PIC18F46K22, configure a UART baud rate, transmit individual characters and strings, receive commands from a computer, use a CP2102 USB-to-TTL adapter, explain why UART TX and RX connections must be crossed, use TXREG1 and RCREG1 correctly, interpret TX1IF and RC1IF, construct structured telemetry messages, implement bidirectional communication, integrate UART with previously learned timer and interrupt concepts, and design firmware that continues operating while communication is occurring. The most important objective is not

simply making text appear on a computer. The objective is understanding how communication becomes another task inside a larger embedded system.

4. Required Hardware

Quantity	Component	Purpose
1	Curiosity HPC with PIC18F46K22	Drone controller
1	CP2102 USB-to-TTL serial adapter	UART-to-USB interface
1	Computer with serial terminal	Ground station
1	LED	ARMED status
1	220 Ω to 470 Ω resistor	LED current limiting
As needed	Jumper wires	Connections

You may use PuTTY, Tera Term, RealTerm, MPLAB X terminal, or another serial terminal capable of opening the CP2102 COM port.

5. UART Configuration

Configure EUSART1 for:

```
Baud Rate:     9600 baud
Data Bits:     8
Parity:        None
Stop Bits:     1
Flow Control:  None
Mode:          Asynchronous
```

This is normally called:

```
9600 8-N-1
```

Both the PIC and the computer must use the same UART configuration. UART does not transmit a clock signal, so the transmitter and receiver must independently agree on the communication timing. For this lab, assume:

```
FOSC = 16 MHz
BRGH = 1
BRG16 = 0
```

Therefore:

```
Baud Rate = FOSC / [16 × (SPBRG + 1)]
```

For 9600 baud:

```
SPBRG = (16,000,000 / (16 × 9600)) - 1

SPBRG ≈ 103
```

Therefore:

```
TXSTA1bits.BRGH = 1;
BAUDCON1bits.BRG16 = 0;
SPBRG1 = 103;
```

Do not blindly copy these values. You should be able to explain during your demonstration why `SPBRG1 = 103` produces approximately 9600 baud.

6. CP2102 Wiring

Use EUSART1 on the PIC18F46K22.

PIC18F46K22	CP2102	Purpose
RC6/TX1	RXD	PIC → Computer
RC7/RX1	TXD	Computer → PIC
GND	GND	Common reference

The wiring is:

```
PIC18F46K22                CP2102

RC6 / TX1 ----------------> RXD

RC7 / RX1 <---------------- TXD

GND ---------------------- GND
```

TX must connect to RX because the output of one device must enter the receiver of the other device. Do not connect the CP2102 VCC or 5 V pin to the Curiosity HPC board unless specifically instructed to power the circuit from the adapter. The Curiosity board should use its normal power source. Also verify the logic levels of the specific CP2102 module before connecting it. The lecture specifically warns that CP2102 boards are available in different configurations and that the pin labeled 5 V does not necessarily describe the UART signal voltage.

7. ARMED Status LED

Connect an external LED to RD0 through a 220 Ω to 470 Ω current-limiting resistor.

```
RD0 ---- Resistor ---->|---- GND
                       LED
```

Use:

```
LATDbits.LATD0 = 0;     // LED OFF
LATDbits.LATD0 = 1;     // LED ON
```

The LED represents propulsion permission only. No motor or propeller is connected in this lab. At reset:

```
State = SAFE
ARMED LED = OFF
```

The controller must never start in the ARMED state.

8. UART Initialization

Create a function:

```
void UART_Init(void);
```

Your function must configure RC6 and RC7, select asynchronous UART operation, configure the baud-rate generator, enable the transmitter, enable the serial port, and enable continuous reception. The important configuration registers are:

```
TXSTA1     Transmitter configuration
RCSTA1     Receiver configuration
BAUDCON1   Baud-rate configuration
SPBRG1     Baud-rate value
```

The UART lecture identifies `SYNC`, `TXEN`, and `BRGH` as the important basic transmitter controls, while `SPEN` and `CREN` enable the serial port and continuous reception. Your initialization should include the appropriate configuration for:

```
TRISCbits.TRISC6
TRISCbits.TRISC7

TXSTA1bits.SYNC
TXSTA1bits.BRGH
TXSTA1bits.TXEN

RCSTA1bits.SPEN
RCSTA1bits.CREN

BAUDCON1bits.BRG16

SPBRG1
```

You must determine and correctly assign the required values.

9. Transmitting One Character

Create:

```
void UART_Write(char data);
```

Before writing another character into TXREG1, your program must verify that the transmit register is ready. The important relationship is:

```
TX1IF = 0  → TXREG1 is not ready for another character

TX1IF = 1  → TXREG1 can accept another character
```

Therefore, the basic transmission operation follows the method introduced in lecture:

```
while(!PIR1bits.TX1IF);

TXREG1 = data;
```

Writing a byte into TXREG1 does not mean the CPU manually creates the Start bit, eight data bits, and Stop bit. The EUSART hardware performs the actual serial transmission.

10. Transmitting a String

UART transmits one character at a time. A telemetry message such as:

```
$STATE,SAFE
```

is therefore transmitted as a sequence of individual characters. Create:

```
void UART_Write_Text(const char *text);
```

Use the same string-transmission concept introduced in lecture:

```
while(*text != '\0')
{
    UART_Write(*text);
    text++;
}
```

The function continues transmitting characters until it reaches the null character marking the end of the C string. Every complete ground-station message should end with:

```
\r\n
```

so the next message begins on a new terminal line.

11. First Communication Test

Before building the complete drone system, verify the physical UART connection. After initialization, transmit:

```
DRONE GROUND LINK ONLINE
```

The terminal should display:

```
DRONE GROUND LINK ONLINE
```

If nothing appears, do not immediately change your program randomly. Debug the system logically. Verify:

```
Is the correct COM port selected?

Is the terminal configured for 9600 8-N-1?

Is PIC TX connected to CP2102 RX?

Is PIC RX connected to CP2102 TX?

Do the PIC and CP2102 share GND?

Is FOSC actually 16 MHz?

Is SPBRG1 correct?
```

```
Is TXEN enabled?

Is SPEN enabled?
```

Do not continue until this test works.

12. Drone Operating State

Create a variable representing the simplified drone state. You may use an enumeration or another clear method. The system requires two states:

```
SAFE
ARMED
```

At startup:

```
state = SAFE
```

and:

```
ARMED LED = OFF
```

When the controller becomes ARMED:

```
ARMED LED = ON
```

When it returns to SAFE:

```
ARMED LED = OFF
```

The UART ground station will control these states.

13. Periodic Drone Telemetry

A real flight controller should not transmit telemetry as quickly as the processor can execute the main loop. Telemetry should be treated as a periodic task. The UART lecture specifically introduces periodic telemetry as part of a real-time architecture rather than continuously flooding the serial connection. Use your previous timer knowledge to generate a telemetry event approximately once every:

```
1 second
```

You may reuse the system-tick architecture developed in the previous labs. Do not implement telemetry timing using:

```
__delay_ms(1000);
```

Your processor must remain available to receive ground-station commands. Every second, transmit two telemetry messages. For example, when SAFE:

```
$STATE,SAFE
$TIME,12
```

and when ARMED:

```
$STATE,ARMED
$TIME,13
```

`TIME` represents the number of seconds since the PIC started running. For example:

```
$STATE,SAFE
$TIME,1
$STATE,SAFE
$TIME,2
$STATE,SAFE
$TIME,3
```

The message format intentionally resembles the telemetry format introduced in lecture, where `$BAT,11800`, `$ALT,245`, and `$STATE,READY` allow the receiver to distinguish the message type from its value. Do not add fake battery or altitude measurements in this lab. You have not learned ADC or the altitude sensor yet. Those telemetry fields will be added in later labs using real measurements.

14. Converting the Time Value to Text

UART sends characters, not an integer variable directly. For example, if:

```
unsigned int seconds = 12;
```

you cannot expect:

```
UART_Write(seconds);
```

to display:

```
12
```

on the terminal. The number must first be represented as ASCII characters. You may write your own simple integer-to-text function or use an appropriate C conversion technique approved by your instructor. Your final transmitted message must appear as:

```
$TIME,12
```

not as an unreadable character. You should be able to explain the difference between the numeric value `12` stored inside the PIC and the ASCII characters `'1'` and `'2'` transmitted through UART.

15. Receiving Ground-Station Commands

Communication must now operate in the opposite direction. The computer will transmit commands to the PIC using the CP2102. For this lab, commands are intentionally one character so you can concentrate on UART operation rather than building a complicated string parser. Use:

```
S = Request Status
A = Arm
D = Disarm
```

Your PIC must detect these characters and perform the required operation.

16. Do Not Block Waiting for UART

The lecture introduced a function such as:

```
char UART_Read(void)
{
    while(!PIR1bits.RC1IF);
    return RCREG1;
}
```

This is useful for learning how UART reception works, but it is not acceptable as the main architecture for this lab. Consider:

```
command = UART_Read();
```

If the ground station sends nothing, the processor remains trapped waiting for a character. Telemetry timing and other flight-controller activities would stop. Instead, your main program must continue running and check whether UART data is available. Conceptually:

```
if(PIR1bits.RC1IF)
{
    command = RCREG1;

    // Process command
}
```

If no UART character has arrived, the processor continues executing the rest of the program. The lecture makes the same distinction: blocking reception may be acceptable for a simple classroom demonstration, but a flight controller must continue sampling sensors, updating timers, monitoring safety conditions, and running other tasks while waiting for communication.

17. Command S — STATUS

When the ground station sends:

```
S
```

the PIC must immediately transmit the current state. If SAFE:

```
$STATUS,SAFE
```

If ARMED:

```
$STATUS,ARMED
```

It should then transmit:

```
$ACK,STATUS
```

For example:

```
$STATUS,SAFE
$ACK,STATUS
```

The periodic telemetry schedule should continue normally after the command is processed.

18. Command A — ARM

When the computer sends:

```
A
```

and the drone is currently SAFE:

```
State → ARMED
ARMED LED → ON
```

The PIC must transmit:

```
$ACK,ARM
$STATE,ARMED
```

The computer should therefore show:

```
$ACK,ARM
$STATE,ARMED
```

The command is only changing the simulated propulsion-permission state. No motor is connected or operated. If the system is already ARMED and another `A` is received, the firmware must remain ARMED and must not behave unpredictably.

19. Command D — DISARM

When the computer sends:

```
D
```

the PIC must immediately enter:

```
SAFE
```

The ARMED LED must turn OFF. Transmit:

```
$ACK,DISARM
$STATE,SAFE
```

The DISARM command must work regardless of whether the system is currently SAFE or ARMED.

20. Invalid Commands

Real communication interfaces must handle unexpected information. If the PIC receives any command other than:

```
S
A
D
```

it must not change the drone state. Instead, transmit:

```
$ERR,CMD
```

For example, if the computer sends:

```
X
```

the response should be:

```
$ERR,CMD
```

The drone must remain in its previous state. This requirement is important. A robust embedded system should fail safely when it receives information that it does not understand.

21. UART Receiver Errors

The PIC18F46K22 provides UART receiver error information through:

```
FERR
OERR
```

`FERR` indicates a framing error, while `OERR` indicates that receive data arrived faster than the software handled it and the receiver overran. These conditions are introduced in the UART lecture. At minimum, your program must recover from an overrun condition so that one communication problem does not permanently disable UART reception. Research the behavior of:

```
RCSTA1bits.OERR
RCSTA1bits.CREN
```

and implement an appropriate recovery method. Be prepared to explain why toggling `CREN` can recover the receiver after an overrun. You do not need to implement an advanced communication-error protocol in this lab.

22. Required Firmware Architecture

Your program should behave conceptually like this:

```
main()
 |
 +---- Configure oscillator
 |
 +---- Configure GPIO
 |
 +---- Configure UART
 |
```

```
+---- Configure timer
|
+---- Initialize state = SAFE
|
+---- ARMED LED OFF
|
+---- Send startup message
|
+---- while(1)
       |
       +---- Check timer
       |       |
       |       +---- Update system time
       |       |
       |       +---- Set telemetry flag every 1 s
       |
       +---- Check UART receiver
       |       |
       |       +---- Character available?
       |               |
       |               +---- Read RCREG1
       |               +---- Process command
       |
       +---- Check UART errors
       |
       +---- Telemetry due?
               |
               +---- Send STATE
               +---- Send TIME
```

The important feature is what is missing:

```
NO long blocking delays
```

and:

```
NO waiting indefinitely for a UART character
```

The main loop must continue circulating.

23. Expected Terminal Operation

A successful demonstration might look similar to:

```
DRONE GROUND LINK ONLINE
$STATE,SAFE
$TIME,1
```

```
$STATE,SAFE
$TIME,2

A

$ACK,ARM
$STATE,ARMED
$STATE,ARMED
$TIME,3
$STATE,ARMED
$TIME,4

S

$STATUS,ARMED
$ACK,STATUS

D

$ACK,DISARM
$STATE,SAFE
$STATE,SAFE
$TIME,5
```

Your exact spacing may differ, but the required message contents and behavior must be clear.

24. Engineering Challenge, Communication Must Not Stop the Drone

The most important test in this lab is not simply whether UART works. Once your program starts, do not type anything into the terminal for at least ten seconds. The PIC must continue transmitting telemetry:

```
$STATE,SAFE
$TIME,1

$STATE,SAFE
$TIME,2

...

$STATE,SAFE
$TIME,10
```

This proves that the processor is not blocked waiting for UART input. Next, send commands at unpredictable times while telemetry is being generated. Commands must still be processed correctly. This demonstrates the embedded-system principle introduced throughout the course:

```
Communication is one task.

It is not the entire program.
```

Later, this same architecture will allow the PIC to communicate with the IMU, barometer, battery monitor, and actuator-control hardware while maintaining the ground-station connection.

25. Required Functions

Organize your program into meaningful functions. At minimum, your program should contain functions equivalent to:

```
void UART_Init(void);

void UART_Write(char data);

void UART_Write_Text(const char *text);

void Send_Telemetry(void);

void Process_Command(char command);
```

You may create additional functions if they improve the organization of your program. Do not place the entire application inside `main()`.

26. Important Restrictions

You must directly configure the PIC18F46K22 EUSART registers. You may not use an external UART software library that hides the peripheral configuration. You must use:

```
EUSART1
RC6/TX1
RC7/RX1
CP2102
9600 baud
8-N-1
```

You must implement bidirectional communication. You must transmit periodic telemetry. You must implement the `S`, `A`, and `D` commands. You must reject invalid commands. You must maintain a safe startup condition. You must not use:

```
__delay_ms(1000);
```

to create the telemetry interval. You must not block the main program waiting indefinitely for UART reception. The objective is to integrate UART into the drone firmware architecture, not simply reproduce the echo example from lecture.

27. Required Demonstration

During the laboratory demonstration, demonstrate the complete system to the instructor. First, reset the PIC and show that the ARMED LED is OFF. Open the computer serial terminal at 9600 baud, 8 data bits, no parity, one stop bit, and no flow control. Show that the startup message appears:

```
DRONE GROUND LINK ONLINE
```

Next, do not send any commands. Show that `$STATE` and `$TIME` telemetry continues to arrive approximately once per second and that the time value continues increasing. Explain why the program does not need to wait for a UART character. Send:

```
S
```

and show that the PIC immediately reports its current state and acknowledges the command. Send:

```
A
```

and show that the controller enters ARMED, the RD0 ARMED LED turns ON, and the ground station receives:

```
$ACK,ARM
$STATE,ARMED
```

Allow the system to continue running and show that periodic telemetry now reports:

```
$STATE,ARMED
```

Send:

```
S
```

again and demonstrate that the returned status is ARMED. Send:

```
D
```

and show that the controller immediately returns to SAFE, the ARMED LED turns OFF, and the terminal reports the new state. Next, send an invalid character such as:

```
X
```

and demonstrate that the drone state does not change and the PIC returns:

```
$ERR,CMD
```

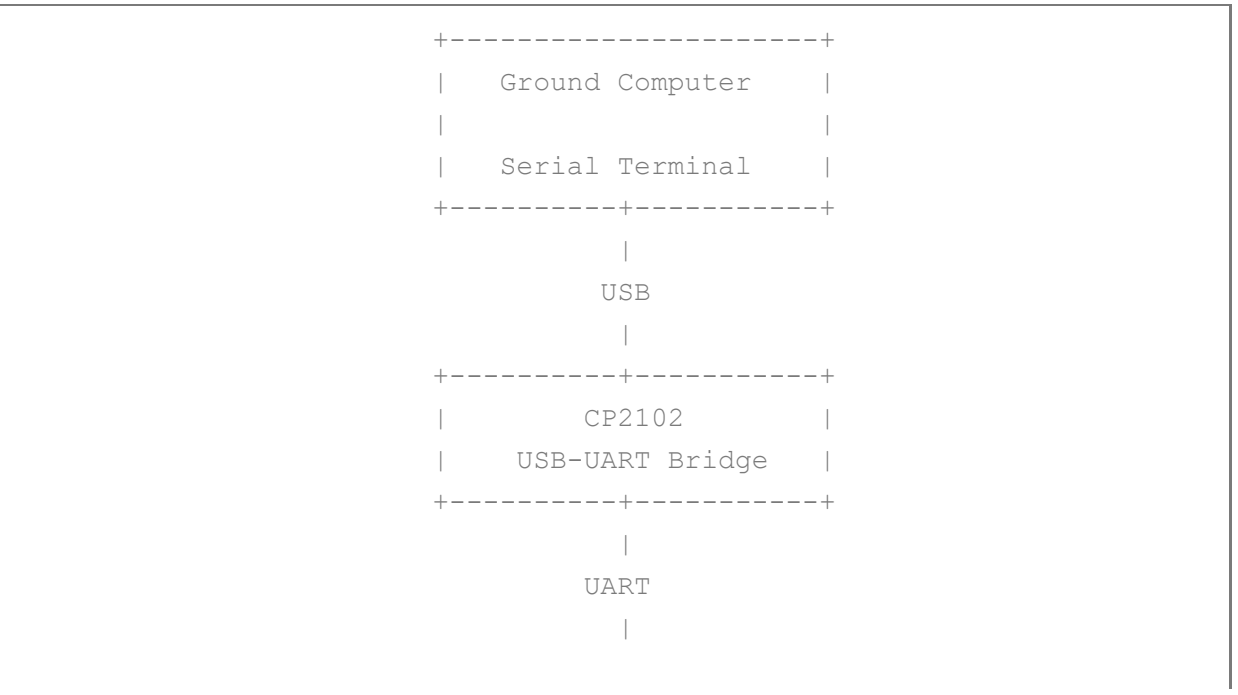
Finally, demonstrate that commands and telemetry can occur together. Send several commands at unpredictable times while the periodic telemetry is running. The program must continue operating correctly without freezing or losing its basic timing behavior.

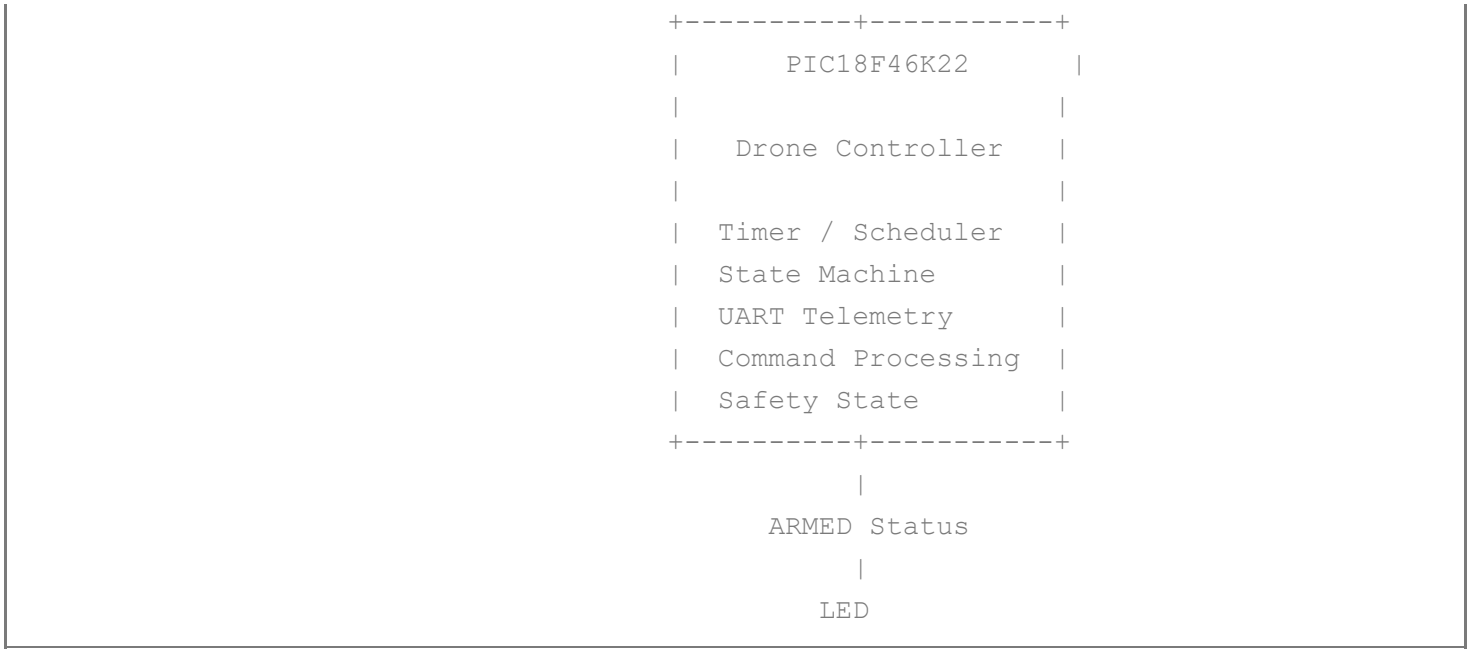
28. What to Submit

Submit your complete MPLAB X project or source code as instructed by your instructor. No formal laboratory report is required. Your code must be readable and appropriately commented. Comments should explain why important operations are performed rather than simply translating every C statement into English. Your program must clearly identify the sections responsible for UART initialization, UART transmission, UART reception, command processing, telemetry generation, system timing, state control, and UART error recovery.

29. Final System

At the end of this lab, your drone controller has gained an important new subsystem:





Earlier labs gave the controller digital inputs and outputs, timing, states, and real-time behavior. This lab gives the controller the ability to communicate with the outside world. The next drone subsystems will begin providing real sensor information. Instead of transmitting simulated status information, future telemetry can contain actual IMU motion, altitude, battery voltage, and eventually propulsion information. The UART interface developed here therefore becomes the debugging and ground-station connection for the rest of the drone project.